Laptop Price Predictor

Machine Learning Regression Project Report

Dataset: Laptop Price Prediction (Kaggle)

Type: Supervised Regression

June 2026

Table of Contents

1. Problem Definition

1.1 Problem Statement

Laptop prices vary widely based on hardware specifications, brand positioning, and build quality, making it difficult for buyers and retailers to quickly estimate a fair price for a given configuration. This project builds a supervised regression model that predicts a laptop's price (in EUR) from its specifications — brand, type, screen, RAM, storage, CPU, GPU, weight, and operating system.

The model is deployed as an interactive Streamlit web application that lets a user enter specifications and instantly see predicted prices from three different machine learning models.

1.2 Input Features

Feature	Type	Description
Company	Categorical	Laptop brand (Dell, Apple, Lenovo, etc.)
TypeName	Categorical	Category — Notebook, Gaming, Ultrabook, etc.
Inches	Numerical	Screen size in inches
Ram	Text → Numerical	RAM, parsed from text e.g. "8GB" → 8
Weight	Text → Numerical	Weight, parsed from text e.g. "1.37kg" → 1.37
ScreenResolution	Text → Multiple	Parsed into width, height, PPI, IPS flag, touchscreen flag
Cpu	Text → Multiple	Parsed into CPU brand and clock speed (GHz)
Gpu	Text → Categorical	Parsed into GPU brand
Memory	Text → Multiple	Parsed into storage type (SSD/HDD/Hybrid) and storage size (GB)
OpSys	Categorical	Operating system, grouped into 5 clean categories

Table 1.1 — Raw input features and how they were derived from the original dataset columns.

1.3 Target Variable

Price_euros — the laptop's price in Euros. The raw distribution is right-skewed (a small number of very expensive laptops), so the model is trained on log_price = log(Price_euros + 1) to stabilize variance, and predictions are converted back with an inverse transform before being shown to the user.

1.4 Why Regression, Not Classification

The target variable, price, is a continuous numeric quantity rather than a discrete category. There is no natural set of price "classes" implied by the problem — buyers want an estimated number, not a label such as "expensive" or "cheap." Regression models (Ridge, Random Forest, XGBoost) are therefore the appropriate model family, evaluated using continuous-error metrics (MAE, RMSE, R^2) rather than classification metrics.

2. Exploratory Data Analysis

A full EDA notebook (Step2_EDA_Laptop_Price.ipynb) was run in Google Colab covering dataset shape, data types, missing values, duplicates, statistical summaries, target distribution, correlation, and outlier analysis, with visualizations for every section.

2.1 Dataset Overview

Property	Value
Rows	~1,303
Raw columns	13
Missing values	Minimal — handled during cleaning
Duplicate rows	Checked and removed during cleaning

Table 2.1 — High-level dataset shape.

2.2 Target Distribution

The Price_euros distribution is right-skewed: most laptops cluster in the budget-to-mid-range, with a long tail of premium and ultra-premium machines. A log transform (log_price) was applied to correct this skew, producing a distribution much closer to normal — this is important because linear models like Ridge Regression assume roughly normally-distributed residuals.

2.3 Correlation & Categorical Analysis

RAM, CPU speed, and screen resolution (via PPI) showed the strongest positive correlation with price. Brand and laptop type (Company, TypeName) showed strong group-wise price differences — for example, Gaming and Workstation laptops, and brands like Apple and Razer, consistently price higher than Notebook-class or budget brands.

2.4 Outlier Analysis

Outliers were identified using the IQR method across RAM, weight, storage, and price. Rather than removing them, outliers were retained, since extremely high-spec and high-price laptops are legitimate data points (not data errors) and removing them would bias the model against correctly pricing premium machines. This decision is documented in the cleaning notebook's decision log.

3. Data Cleaning

Cleaning was performed in Step3_Data_Cleaning_Laptop_Price.ipynb. Every decision below is logged in that notebook alongside before/after comparisons.

3.1 Missing Value Treatment

The dataset had minimal missing values. Numerical columns were imputed using the median (robust to outliers); categorical columns were imputed using the most frequent value. This logic was implemented as part of the preprocessing pipeline (Section 5) rather than only at the cleaning stage, so it generalizes correctly to new, unseen input at prediction time.

3.2 Duplicate Removal

Duplicate rows were checked for and removed where found, to prevent the model from over-weighting repeated observations during training.

3.3 Outlier Handling

As discussed in Section 2.4, outliers were retained rather than removed or capped, since they reflect genuine premium-laptop pricing rather than data entry errors. The right-skew this causes in the target variable was instead addressed via log transformation.

3.4 Data Consistency Checks

Six sanity checks were run after cleaning, including verifying that parsed numeric fields (RAM, weight, storage, resolution) fall within plausible real-world ranges, that categorical fields contain no unexpected values, and that the row count after cleaning matches expectations (no accidental data loss).

3.5 Text Parsing

Raw Column	Example Value	Parsed Into
Ram	"8GB"	Ram_GB = 8
Weight	"1.37kg"	Weight_kg = 1.37
ScreenResolution	"1920x1080"	res_width, res_height, ppi, is_touchscreen, is_ips
Cpu	"Intel Core i7 8th Gen"	cpu_brand, cpu_ghz
Gpu	"Nvidia GeForce GTX 1050"	gpu_brand
Memory	"256GB SSD"	storage_type, storage_gb
OpSys	"Windows 10"	os_clean (grouped into 5 categories)

Table 3.1 — Text fields parsed into structured numeric/categorical features.

4. Feature Engineering

Seven engineered features were created in Step4_Feature_Engineering_Laptop_Price.ipynb, covering feature creation, binning, and interaction techniques — well beyond the assignment's minimum of three.

Feature	Technique	Why It Was Created
log_price	Log Transform	Corrects right-skew in the target variable, improving linear model assumptions
ppi	Feature Creation	Pixel density (display sharpness) derived from resolution ÷ screen size — a 13" 1080p screen is sharper than a 17" 1080p screen, which a model can't infer from resolution alone
ram_tier	Binning	Groups RAM into market-recognizable tiers (Low/Mid/High/Ultra) to capture non-linear pricing jumps between tiers
storage_tier	Binning	Groups storage into tiers (128/256/512GB/1TB+), similarly capturing non-linear price jumps
cpu_perf_score	Interaction Feature	Combines CPU brand quality weight × clock speed into one composite performance signal
is_premium	Feature Creation (composite)	Binary flag combining brand, laptop type, RAM, and storage to flag likely premium machines
weight_category	Binning	Groups weight into Ultralight/Light/Standard/Heavy classes, a marketing-relevant grouping

Table 4.1 — Engineered features, the technique used, and the reasoning behind each.

5. Data Preprocessing Pipeline

Implemented in Step5_Preprocessing_Laptop_Price.ipynb using scikit-learn's ColumnTransformer, ensuring the exact same transformation logic is applied at training time and prediction time.

5.1 Pipeline Steps

1. Train-test split — 80/20, random_state=42 for reproducibility.
2. Numerical features — median imputation, followed by StandardScaler.
3. Categorical features — most-frequent imputation, followed by One-Hot Encoding (unknown categories at inference time are safely ignored).
4. Binary features — passed through unchanged.
5. Feature selection — SelectKBest (f_regression) reduces the expanded feature set down to the top 20 most predictive features, reducing noise and overfitting risk.
6. Leakage check — verifies the preprocessor is fit only on training data, never on test data, before being applied to both.

5.2 Why This Order Matters

Fitting the scaler and encoder only on the training split (not the full dataset) prevents information from the test set leaking into preprocessing statistics — a common and subtle source of inflated model performance. The same fitted preprocessor object is reused for every new prediction made through the Streamlit app, guaranteeing consistency between training and deployment.

6. Model Training

Three regression models were trained and compared, satisfying the assignment's requirement to train at least three models with justification (Step6_Model_Training_Laptop_Price.ipynb).

6.1 Models & Justification

Model	Key Hyperparameters	Why It Was Chosen
Ridge Regression	alpha = 1.0	Linear baseline with L2 regularization; handles multicollinearity introduced by one-hot encoded categorical features well, and serves as an interpretable lower bound for performance
Random Forest	n_estimators=200, max_depth=12, min_samples_split=5	Captures non-linear relationships and feature interactions (e.g. RAM × premium brand) without manual specification; robust to outliers retained during cleaning
XGBoost	n_estimators=300, learning_rate=0.05, max_depth=6, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1, reg_lambda=1.0	Gradient boosting with both L1 and L2 regularization; consistently among the strongest performers on structured/tabular regression tasks, and includes built-in feature importance

Table 6.1 — All three trained models, configuration, and selection rationale.

6.2 Cross-Validation

5-fold cross-validation was used during training to assess model stability across different train/validation splits, in addition to the single held-out 20% test set used for final evaluation.

6.3 Live Deployment Behavior

The deployed Streamlit application retrains all three models fresh from the feature-engineered dataset on every cold start (cached for the session afterward), rather than loading serialized .pkl files. This design choice was made to eliminate scikit-learn/XGBoost version-mismatch errors between the Google Colab training environment and the Streamlit Cloud deployment environment, at the cost of a one-time ~30–45 second startup delay. Serialized model.pkl and preprocessor.pkl files (produced via joblib, see Section 8) are still included in the repository as required deliverables and were verified to load and predict correctly in the training environment.

7. Model Evaluation

Evaluated in Step7_Model_Evaluation_Laptop_Price.ipynb using the regression metrics required by the assignment: MAE, MSE, RMSE, and R^2, plus MAPE for additional interpretability. All metrics below are computed on log_price; R^2 and MAPE are scale-independent and directly comparable.

7.1 Model Comparison

Model	R^2 (Test)	MAE	Notes
Ridge Regression	Lower of the three	Higher	Best when interpretability matters; underfits non-linear pricing patterns
Random Forest	Mid-range	Mid-range	Strong general-purpose performer; less prone to overfitting than boosting if undertuned
XGBoost	Highest of the three	Lowest	Best overall fit; selected as the primary/best model based on test R^2

Table 7.1 — Relative model performance pattern observed in this project. Exact numeric values are produced fresh on each training run and displayed live in the Streamlit app and in Step7's notebook output.

7.2 Residual & Overfit Analysis

Residual plots and train-vs-test R^2 comparisons were used to check for overfitting in the tree-based models. Actual-vs-predicted scatter plots were generated for all three models, and error was additionally segmented by price range to check whether the model performs worse for high-priced laptops (a common failure mode caused by their relative scarcity in the training data).

7.3 Final Model Selection

Across the held-out test set, XGBoost achieved the strongest combination of R^2 and MAE, consistent with its design advantages (gradient boosting with regularization) on this kind of tabular, mixed-type feature set. It is presented as the headline "best model" in the deployed app, while Ridge Regression and Random Forest are shown alongside it for transparency and comparison, directly satisfying the requirement to train and justify at least three models.

8. Model Serialization

Implemented in Step8_Model_Serialization_Laptop_Price.ipynb. All artifacts were saved using joblib.dump() as required, with an end-to-end pipeline test (raw input → preprocess → select → predict → currency conversion) verified after reloading every file from disk.

File	Purpose
models/model.pkl	Best-performing trained model (XGBoost)
models/preprocessor.pkl	Fitted ColumnTransformer (imputation, scaling, encoding)
models/feature_selector.pkl	Fitted SelectKBest feature selector
models/metadata.json	Column lists and model metadata used by the app
models/selected_features.csv	Names of the 20 selected features

Table 8.1 — Serialized artifacts and their purpose.

9. Streamlit Integration

The app (app.py) provides a full interactive prediction experience meeting every requirement in the assignment brief.

Requirement	Implementation
User input form	Sidebar with all 15 specification inputs (brand, type, screen, RAM, storage, CPU, GPU, OS, weight)
Prediction button	"Predict Price" button triggers inference
Prediction output	Predicted price shown in INR (with EUR reference), price range, and market segment label, for all 3 trained models
Error handling	Try/except blocks around model loading and prediction, with user-facing error messages
Basic UI styling	Custom CSS — gradient header, styled prediction cards, info cards for spec summary

Table 9.1 — Streamlit app requirements and how each was satisfied.

9.1 Live Application

https://laptop-price-predictor-emazcbznuo69tzqsbqfbnu.streamlit.app/ →

10. Repository Structure

The GitHub repository follows the structure required by the assignment:

```
laptop-price-predictor/
├── data/                       # Feature-engineered dataset
├── notebooks/                  # All pipeline notebooks (Steps 2–8)
├── src/                        # Reserved for reusable source modules
├── models/
│   ├── model.pkl
│   └── preprocessor.pkl
├── app.py                      # Streamlit application
├── requirements.txt
├── README.md
└── report.pdf                  # This document
```

11. Bonus Opportunities (Not Implemented)

The following bonus items from the assignment brief were not implemented in this submission and are noted here as potential future extensions:

- Dockerizing the Streamlit application for environment-independent deployment
- Deployment on Render as an alternative/additional hosting platform
- CI/CD automation using GitHub Actions
- Experiment tracking with MLflow across model training runs
- SHAP-based explainability for individual predictions

12. Conclusion

This project delivers a complete, end-to-end machine learning pipeline for laptop price prediction — from raw, messy text fields through structured feature engineering, a leak-free preprocessing pipeline, three trained and compared regression models, full evaluation, serialization, and a live, publicly deployed Streamlit application. XGBoost was identified as the strongest-performing model on held-out data, while Ridge Regression and Random Forest are retained in the deployed app for direct, transparent comparison — directly satisfying the assignment's model-training and justification requirements.